Training Neural Networks with Keras

Neural networks and computer vision

ECE 116 | Fall 2026

Adapted from Farias and Ramakrishnan, MIT OpenCourseWare 15.773, Spring 2024. Changes were made. CC BY-NC-SA 4.0.

Lightning Intro to Tensorflow/Keras

What’s a Tensor?

What’s a Tensor?

Tensor of rank 0 (Scalar)

42

What's a Tensor?

Tensor of rank 0 (Scalar)

42

Tensor of rank 1 (aka Vector)

(42, 23.4, 11.2)

What's a Tensor?

Tensor of rank 0 (Scalar)

42

Tensor of rank 2 (aka Matrix)

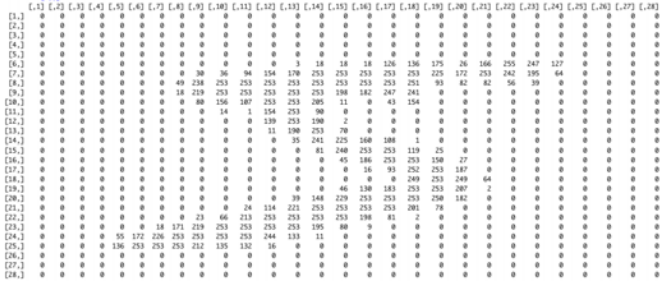


Image credit: fast.ai

Tensor of rank 1 (aka Vector)

(42, 23.4, 11.2)

What's a Tensor?

Tensor of rank 0 (Scalar)

42

Tensor of rank 1 (aka Vector)

(42, 23.4, 11.2)

Tensor of rank 2 (aka Matrix)

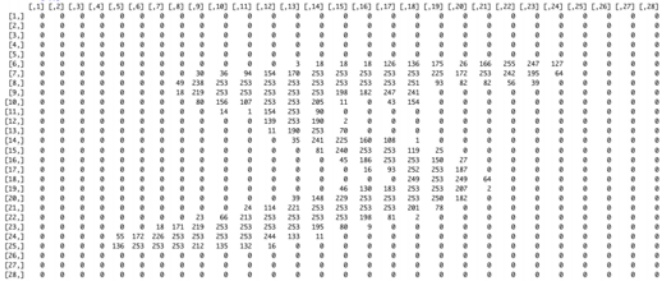
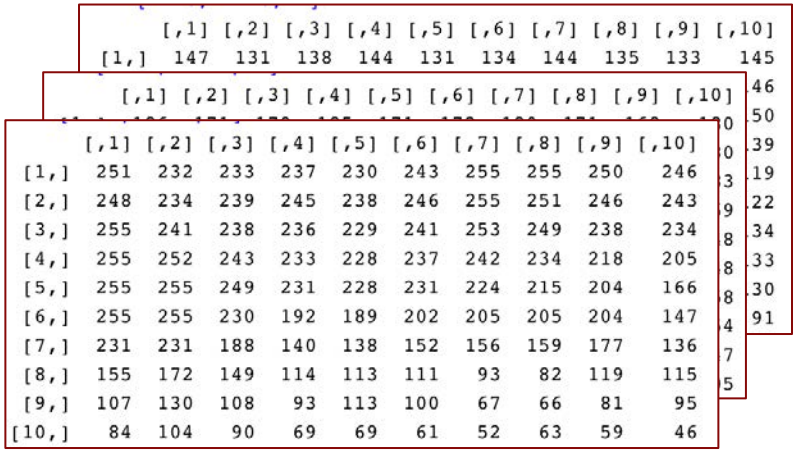


Image credit: fast.ai

Tensor of rank 3 (aka "cube")



	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]	[,9]	[,10]
[1,]	251	232	233	237	230	243	255	255	250	246
[2,]	248	234	239	245	238	246	255	251	246	243
[3,]	255	241	238	236	229	241	253	249	238	234
[4,]	255	252	243	233	228	237	242	234	218	205
[5,]	255	255	249	231	228	231	224	215	204	166
[6,]	255	255	230	192	189	202	205	205	204	147
[7,]	231	231	188	140	138	152	156	159	177	136
[8,]	155	172	149	114	113	111	93	82	119	115
[9,]	107	130	108	93	113	100	67	66	81	95
[10,]	84	104	90	69	69	61	52	63	59	46

Can you give an example of a rank-4 tensor?

Tensorflow

Tensorflow (TF) is a library that provides

- Automatic calculation of the gradient of (complicated) loss functions

$$\nabla Loss(w) = \left[\frac{\partial Loss}{\partial w_1}, \frac{\partial Loss}{\partial w_2}, \ldots, \frac{\partial Loss}{\partial wn}\right]$$

Tensorflow

Tensorflow (TF) is a library that provides

- Automatic calculation of the gradient of (complicated) loss functions
- Library of state-of-the-art optimizers

Tensorflow

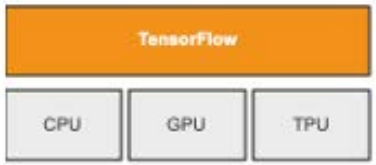
Tensorflow (TF) is a library that provides

- Automatic calculation of the gradient of (complicated) loss functions
- Library of state-of-the-art optimizers
- Automatic distribution of computational load across servers

Tensorflow

Tensorflow (TF) is a library that provides

- Automatic calculation of the gradient of (complicated) loss functions
- Library of state-of-the-art optimizers
- Automatic distribution of computational load across servers
- Automatic adaptation of code to work on parallel hardware (GPUs and TPUs)



Keras “sits on top of” Tensorflow …

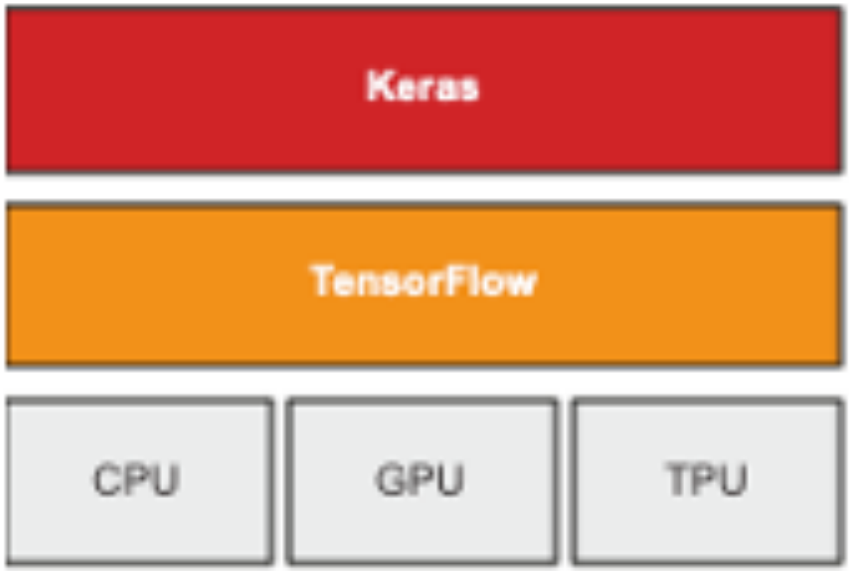


Image: Page 70 of textbook

… and provides “convenience” features

- Pre-defined layers
- Incredibly flexible ways to specify network architectures
- Easy ways to preprocess data
- Easy ways to train models and report metrics
- Pre-trained models you can download and customize

Keras model-building APIs

- Sequential: a linear stack of layers
- Functional API: a graph of connected layers
- Subclassing: a custom model class

ECE116 uses the Functional API.

Check out the wealth of introductory and advanced material, with accompanying colabs, at tensorflow.org and keras.io

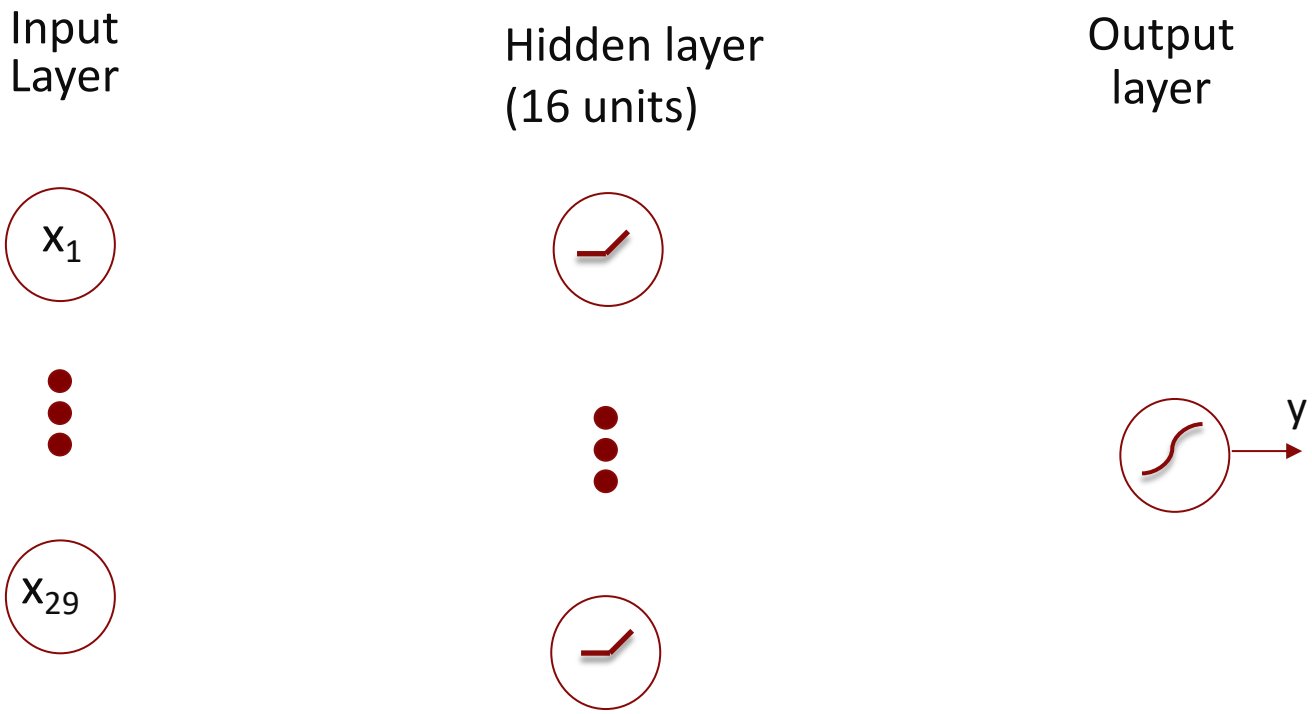
Building the Model in Keras

ECE116 lecture section

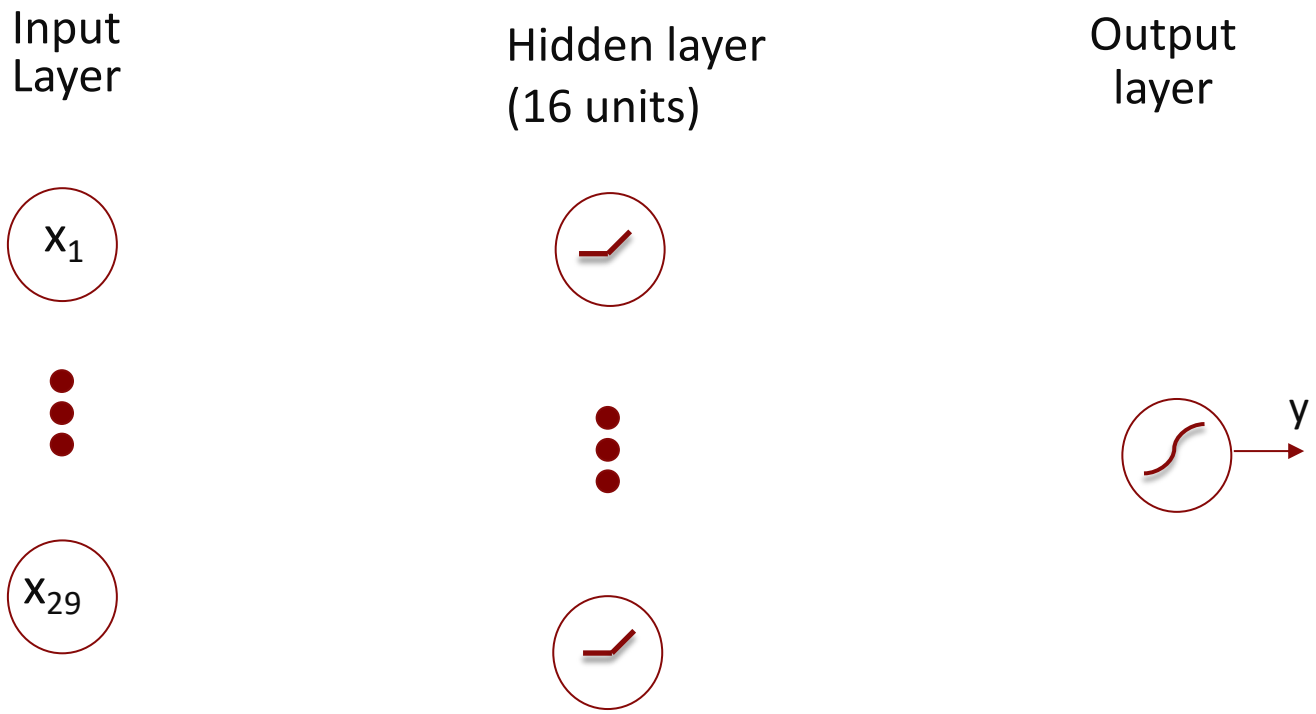
The network maps directly to Keras code

- Each network layer becomes a Keras layer declaration.
- The output of one layer becomes the input to the next.

Typically, we define each layer from left to right

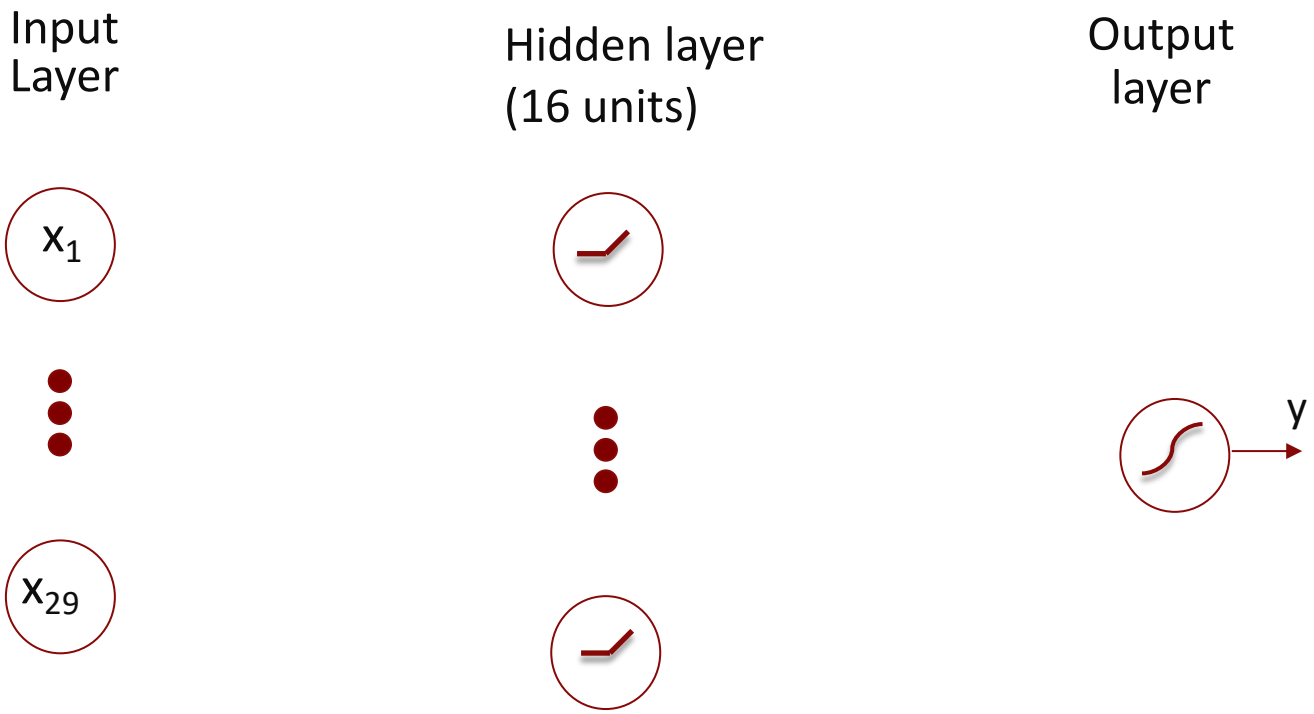


Let's start with the input layer



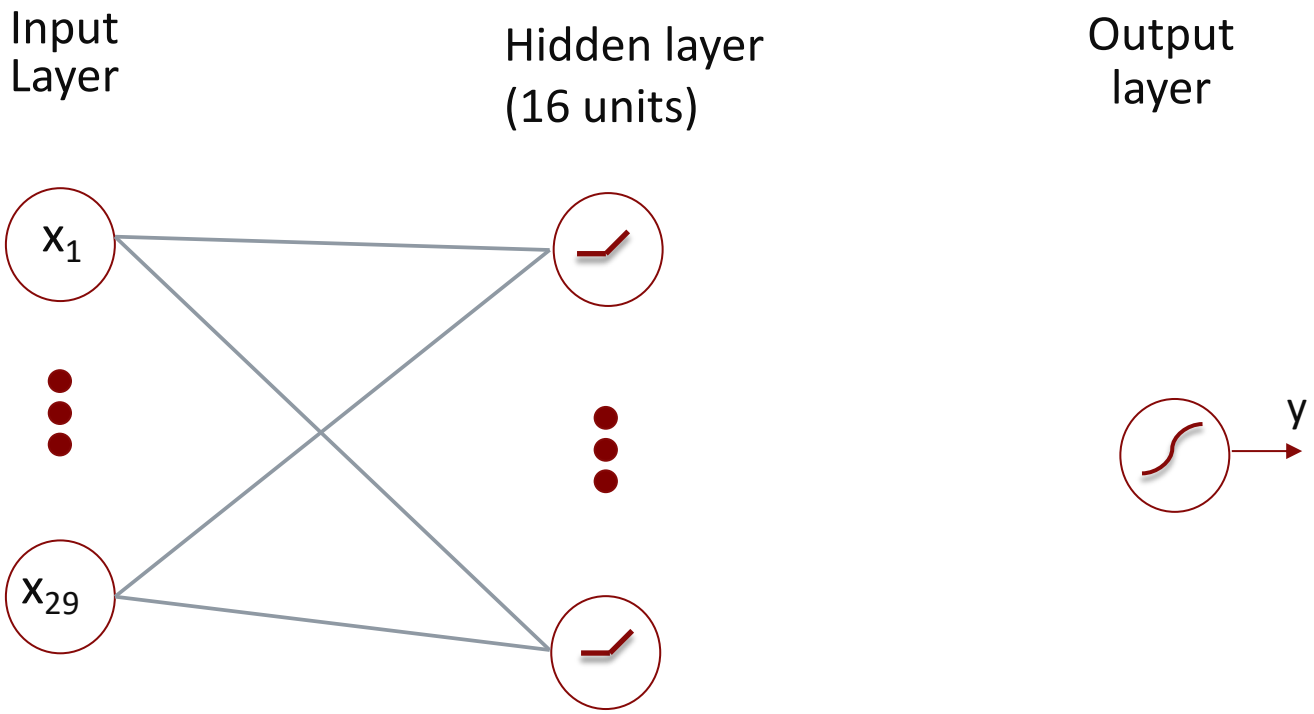

```
input = keras.Input(shape=29)
```

We specify the shape of the input




```
input = keras.Input(shape=29)
```

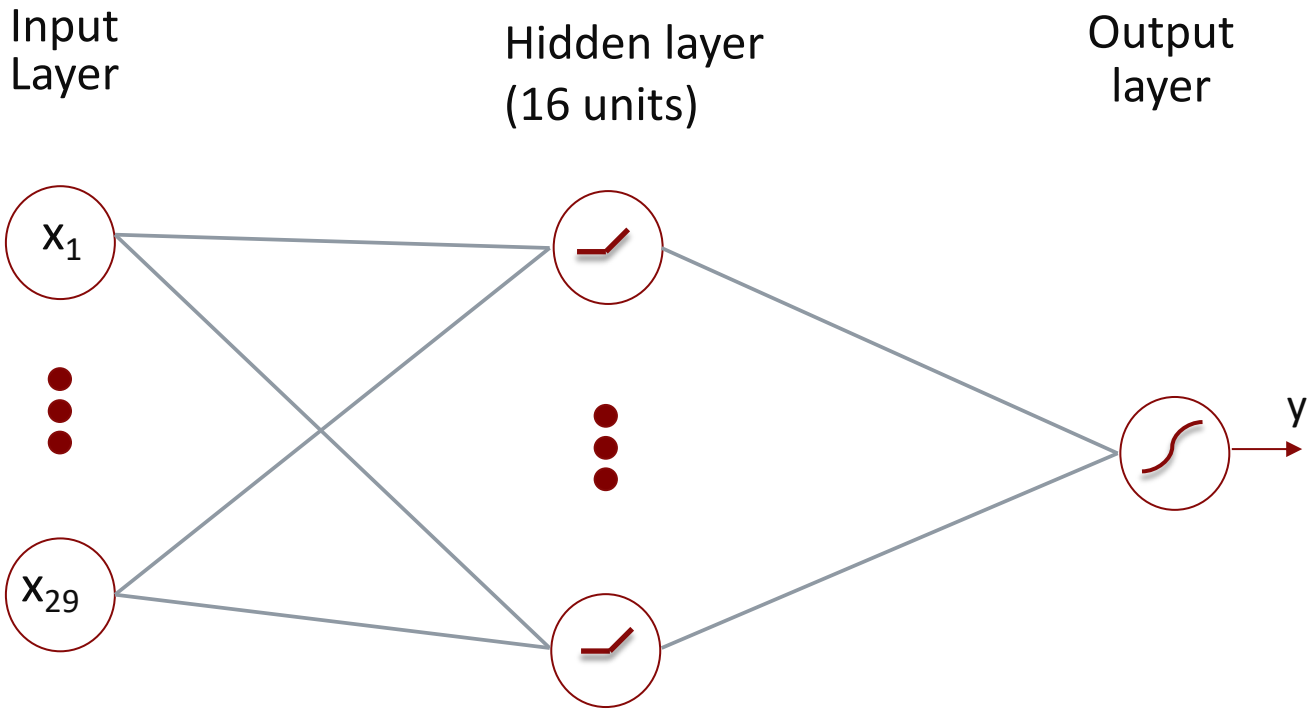
Create and connect the hidden layer




```
input = keras.Input(shape=29)
```

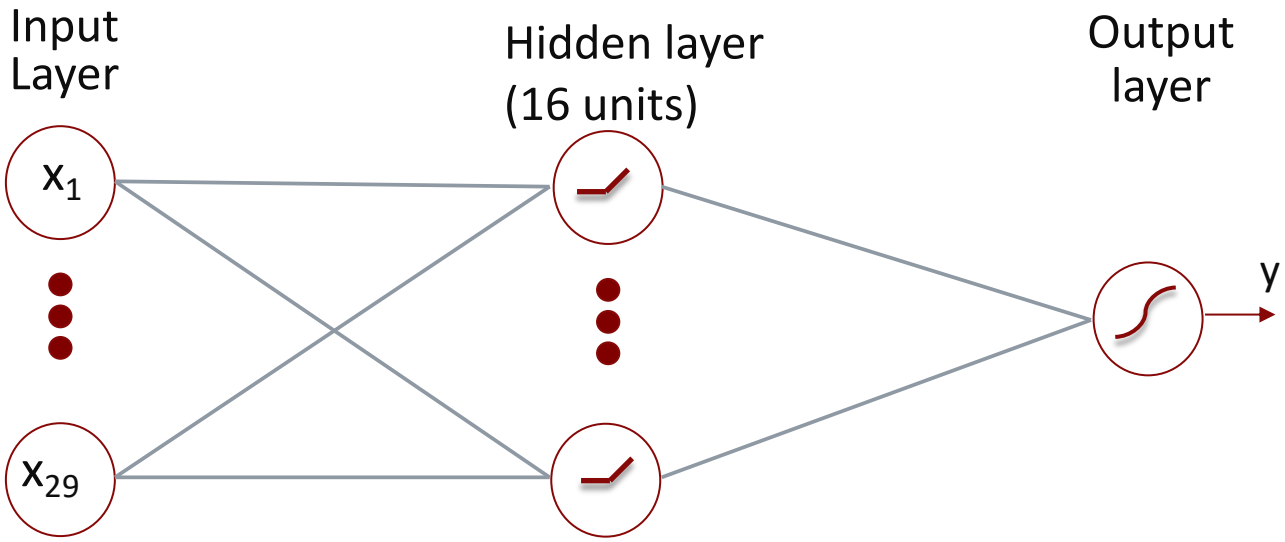
```
h = keras.layers.Dense(16, activation="relu")(input)
```

Create and connect the output layer



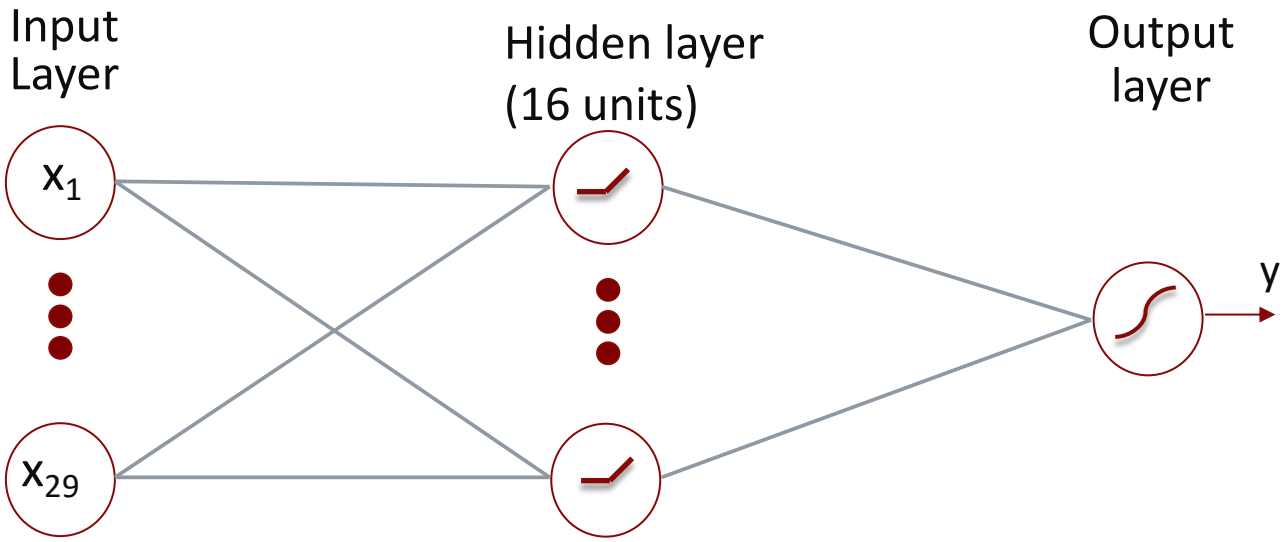

```
input = keras.Input(shape=29)
   h = keras.layers.Dense(16, activation="relu")(input)
output = keras.layers.Dense(1, activation="sigmoid")(h)
```

We have defined and connected the layers




```
input = keras.Input(shape=29)
h = keras.layers.Dense(16, activation="relu")(input)
output = keras.layers.Dense(1, activation="sigmoid")(h)
```

The final step is to formally define a model.




```
input = keras.Input(shape=29)
h = keras.layers.Dense(16, activation="relu")(input)
output = keras.layers.Dense(1, activation="sigmoid")(h)
model = keras.Model(input, output)
```

That's it!

A Neural Model for Heart Disease Prediction

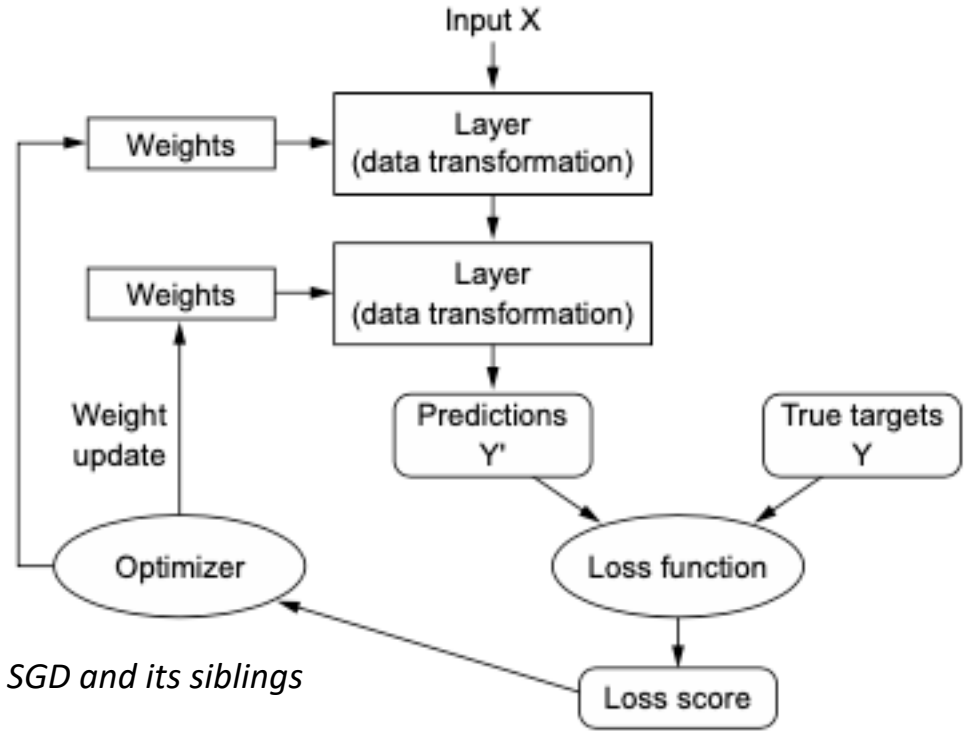
```
input = keras.Input(shape=29)
h = keras.layers.Dense(16, activation="relu")(input)
output = keras.layers.Dense(1, activation="sigmoid")(h)
model = keras.Model(input, output)
```

We will show how to train this model with real data and use it for prediction after we cover some *conceptual* building blocks

Training Mechanics

ECE116 lecture section

(Recap) Summary of overall training flow



SGD and its siblings

Figure 2.26 Relationship between the network, layers, loss function, and optimizer

Image: Page 61 of textbook (Deep Learning with Python, Francois Chollet, 2nd Edition)

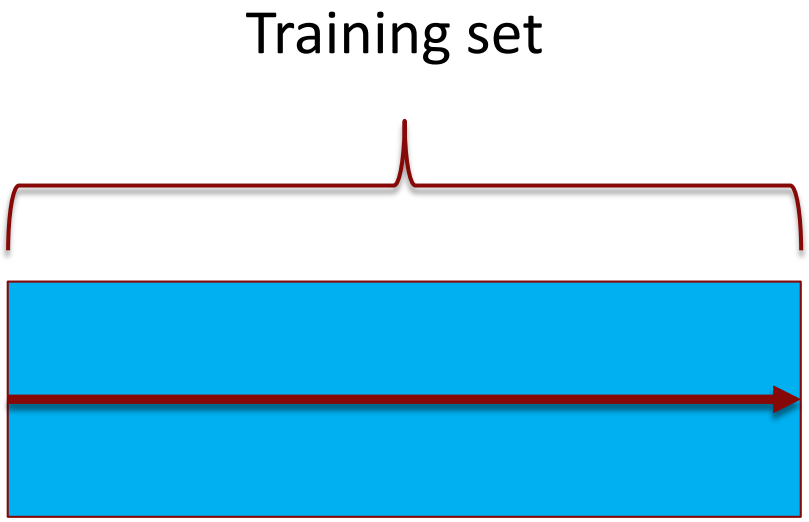
(Recap) Gradient Descent vs Stochastic Gradient Descent

- At each iteration, use all data points to calculate the gradient of the loss function

- At each iteration, randomly choose just a few of the data points and use only these to compute the gradient of the loss function

Epochs and Batches

What is an epoch?



An epoch is one pass through the full training set.

But this plays out differently for Gradient Descent vs *Stochastic* Gradient Descent.

An epoch in Gradient Descent

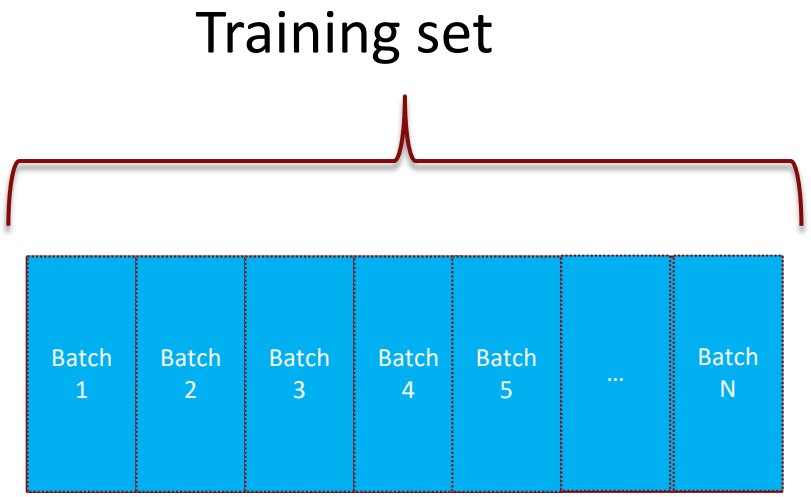


- We run every training sample through the network to get the predictions
- We calculate the gradient of the loss
- We update the parameters

$$w \leftarrow w - \alpha \frac{dLoss(w)}{dw}$$

This is done just once at the end of the epoch

An epoch in Stochastic Gradient Descent

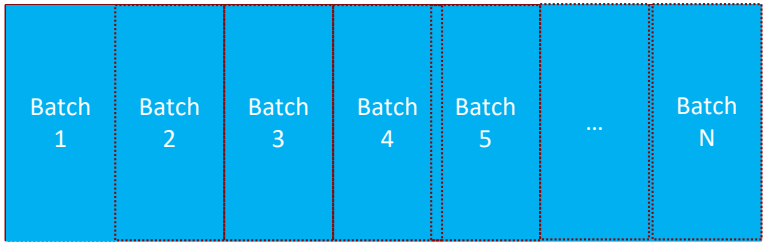


But when we do Stochastic Gradient Descent (SGD), we process the data in minibatches, one after the other*

*we will refer to minibatches as batches from now on for simplicity

An epoch in Stochastic Gradient Descent

Training set

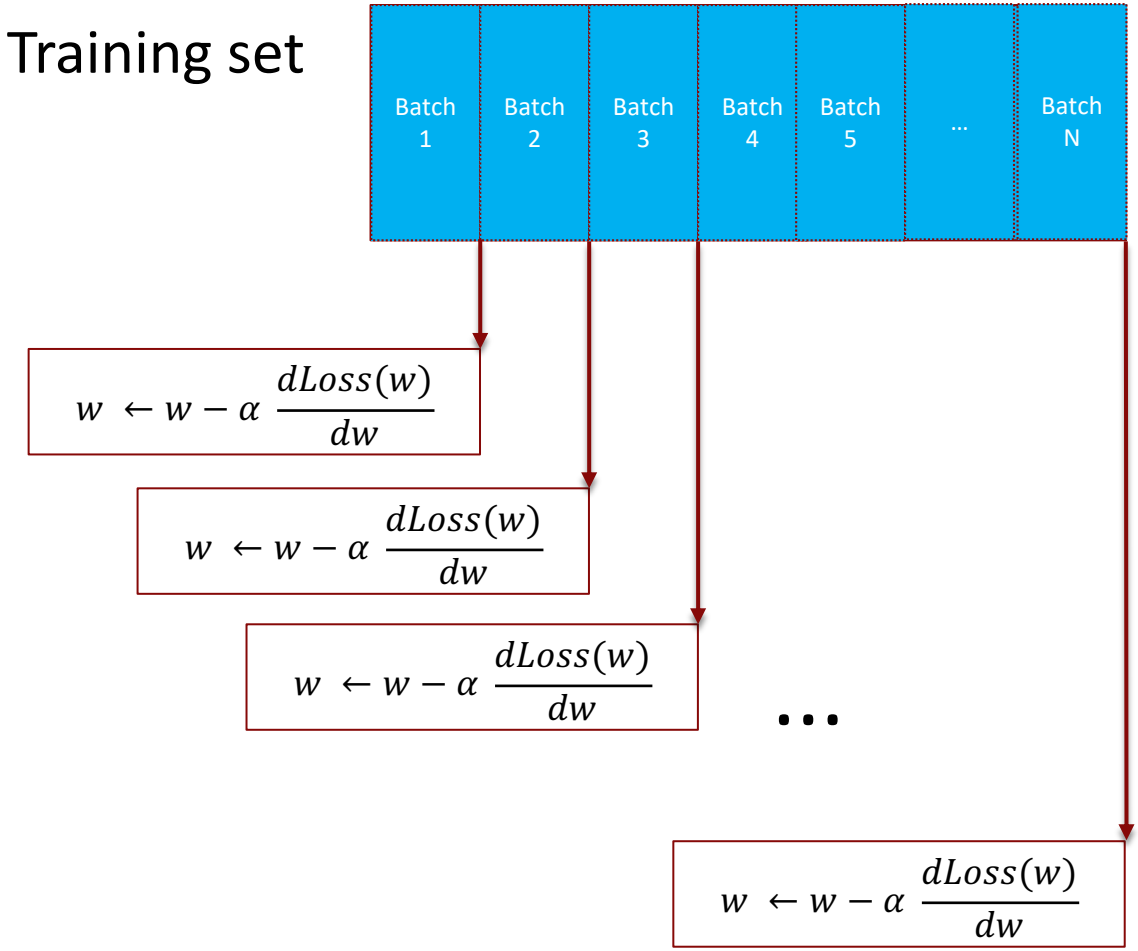


For each batch:

- *We run the training samples in that batch through the network to get predictions*
- *We calculate the gradient of the loss*
- *We update the parameters*

$$w \leftarrow w - \alpha \frac{dLoss(w)}{dw}$$

An epoch in Stochastic Gradient Descent

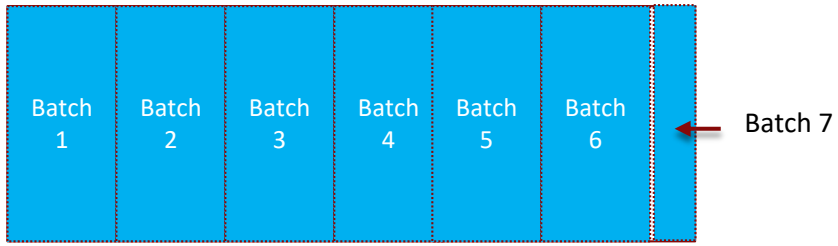


How many batches in an epoch when we do SGD?

of batches in one epoch = (Training set size / Batch size) rounded up

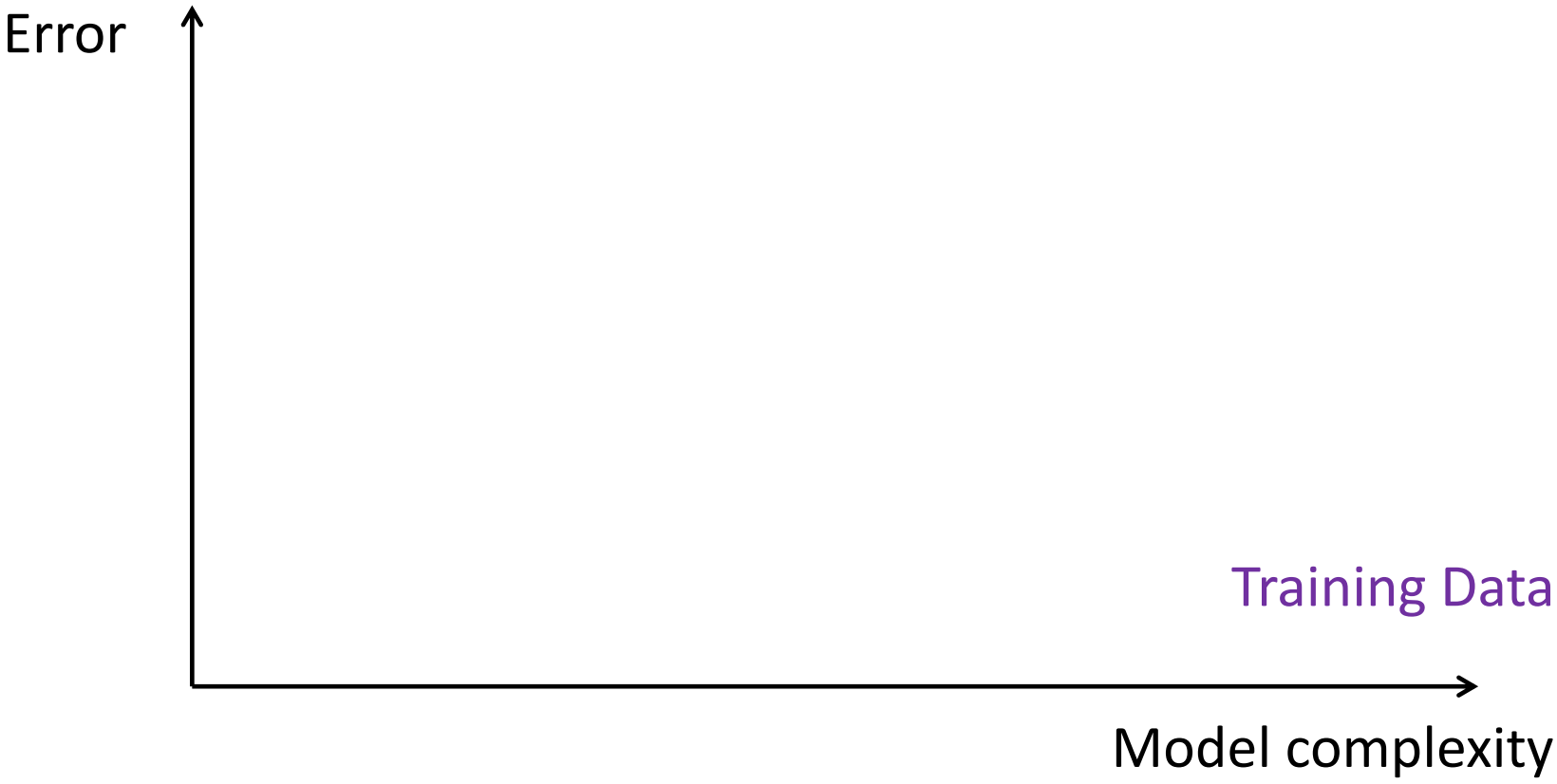
For Neural Heart Disease Model:
Training set size = 194
Batch size = 32
of batches in one epoch = (194/32) rounded up = 7

The first 6 batches have 32 samples each, and the 7th batch has the last 2 samples.
32 * 6 + 2 = 194

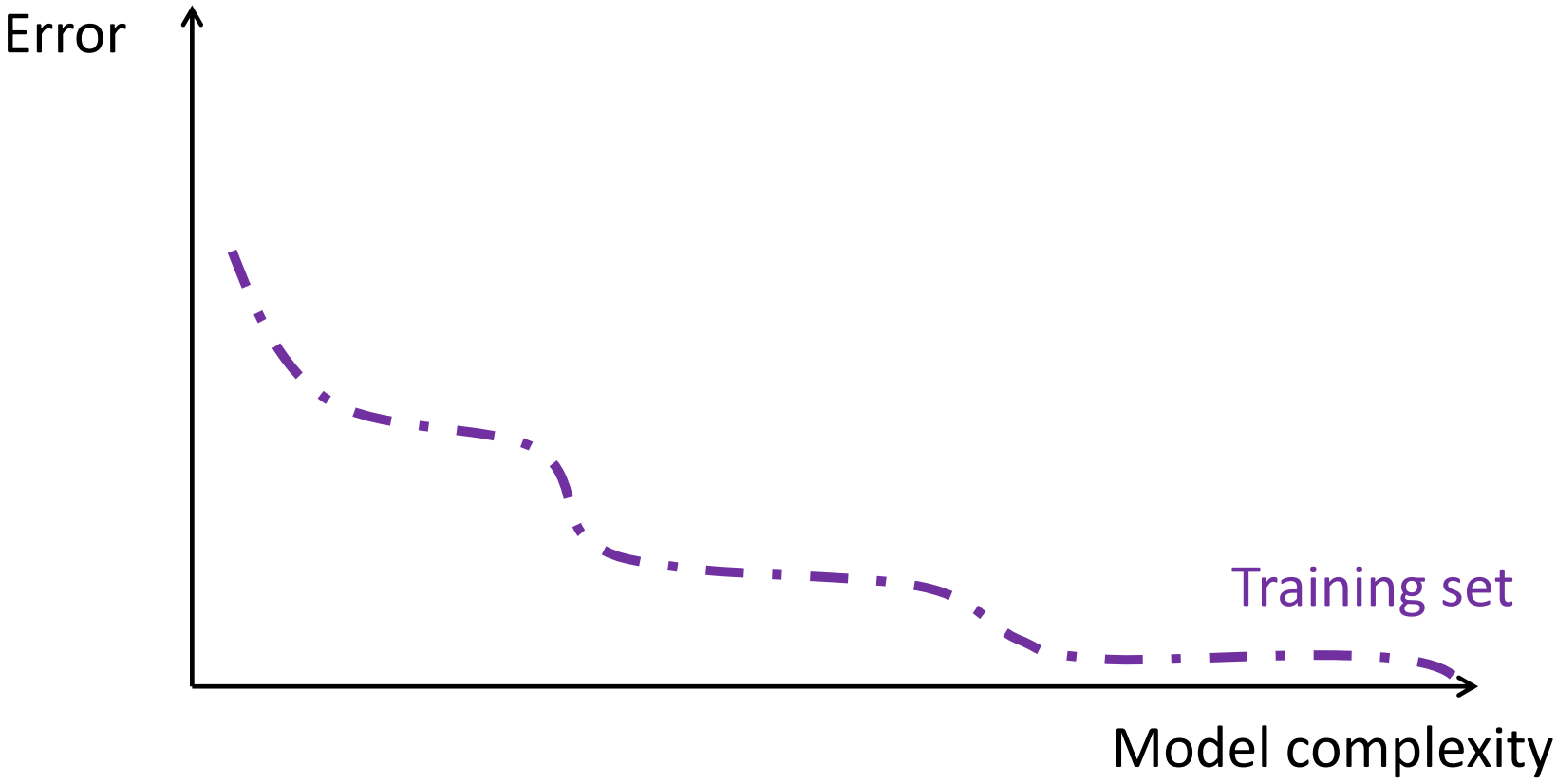


Overfitting and Regularization

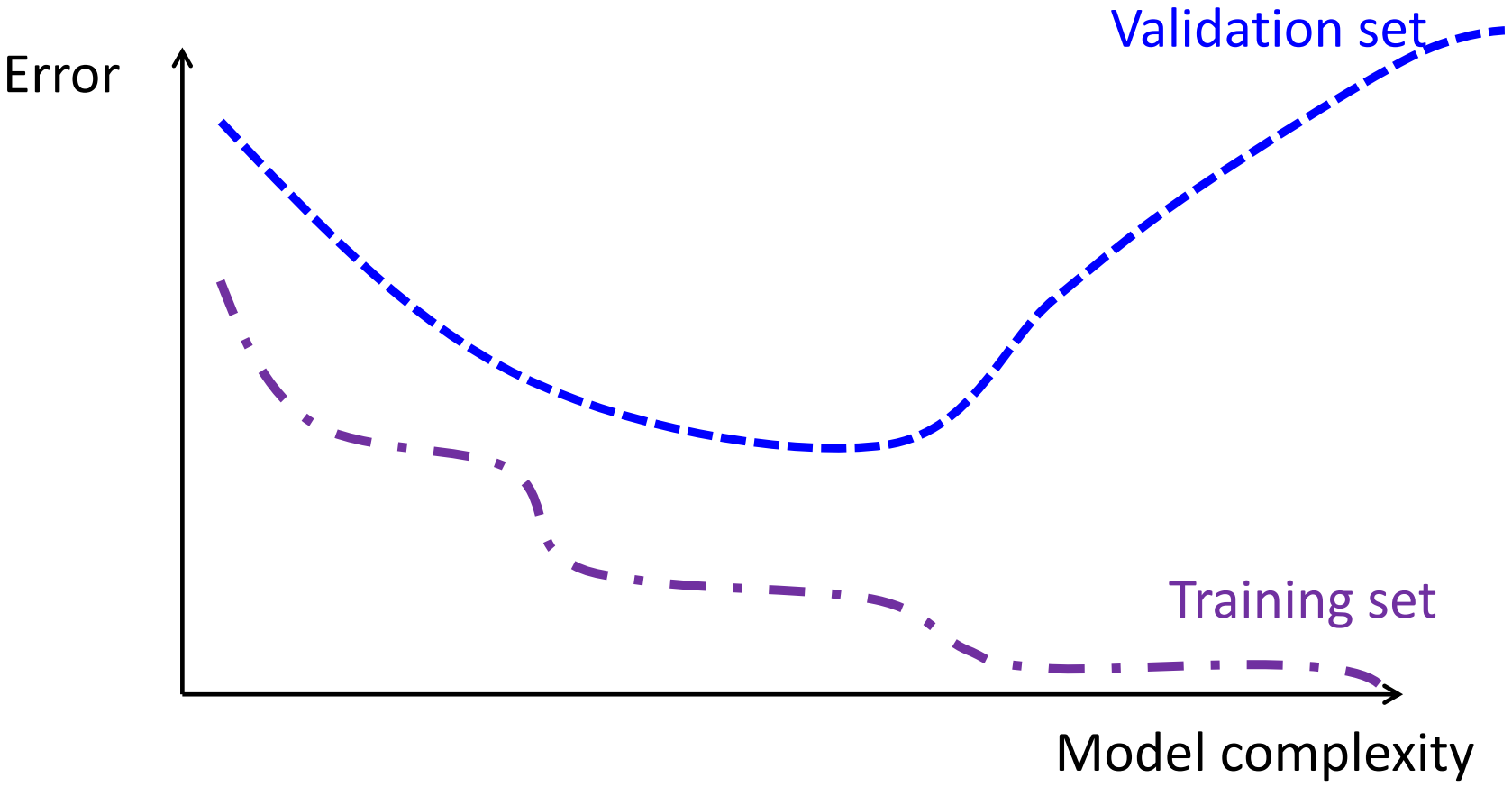
Recall Underfitting vs. Overfitting



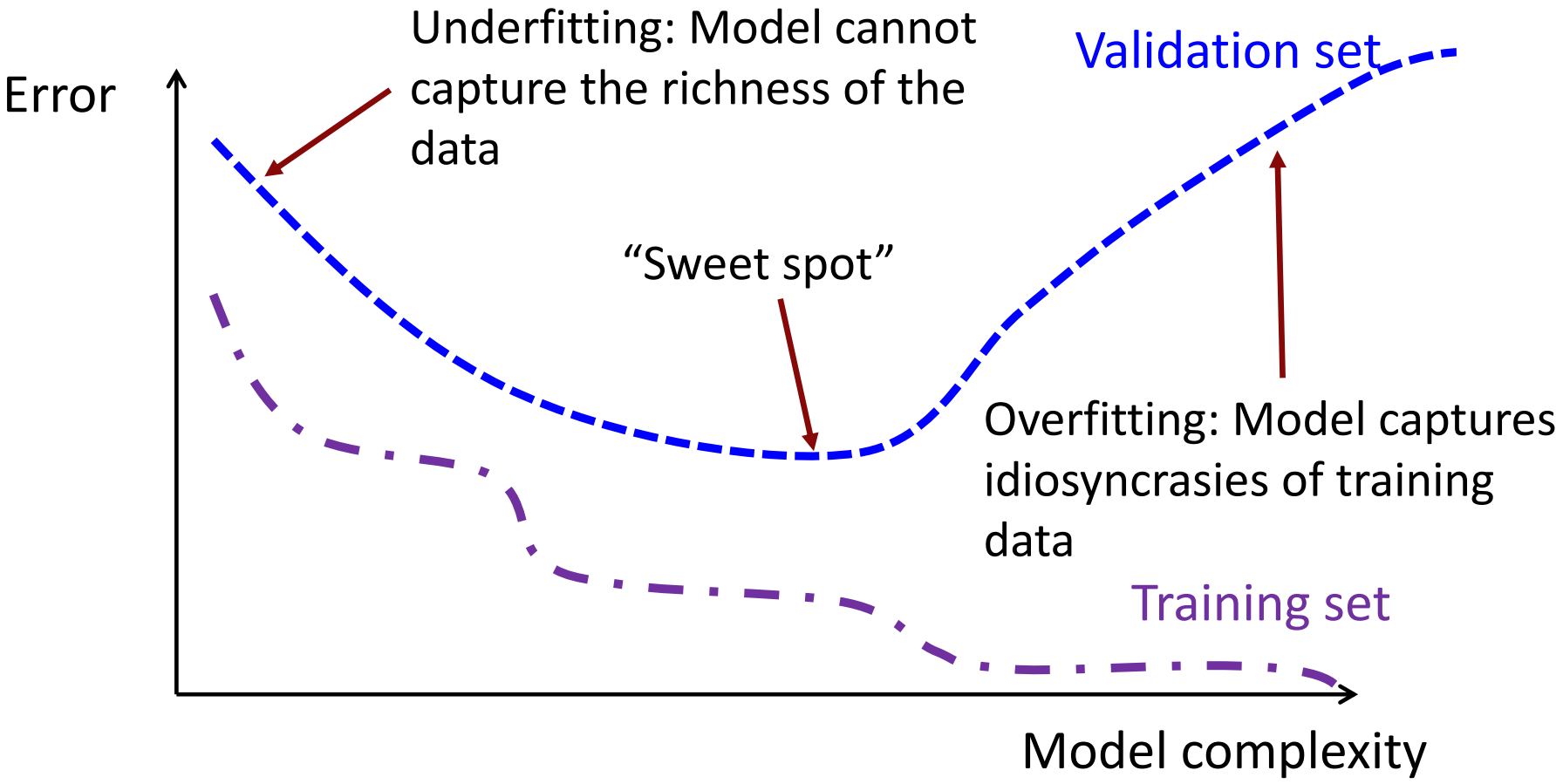
Recall Underfitting vs. Overfitting



Recall Underfitting vs. Overfitting



Recall Underfitting vs. Overfitting

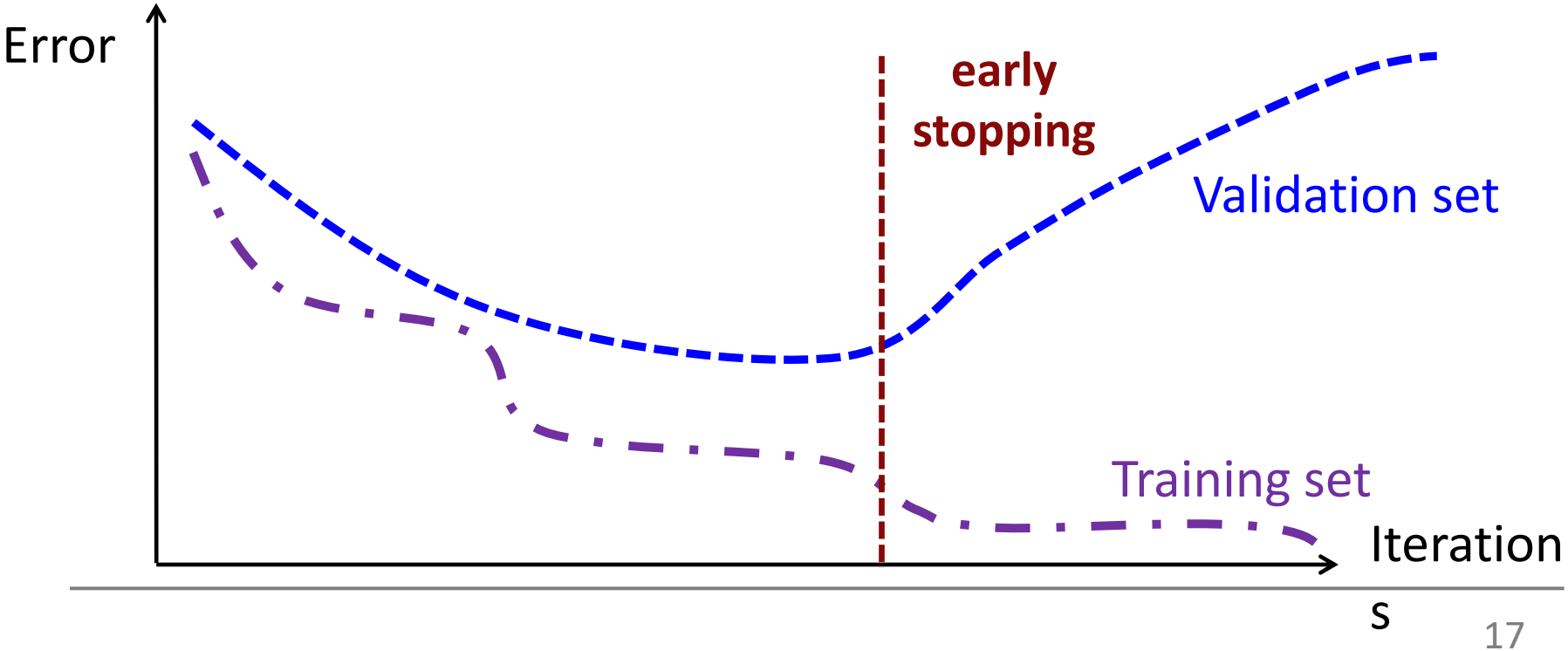


Overfitting in Neural Networks

- To learn smart representations of complex, unstructured data, the NN needs to have large “capacity” i.e., many layers and many neurons in each layer
- But this raises the likelihood of overfitting so we need to add *regularization*
- Several regularization methods have been developed to address this problem

Regularization strategy: *Early Stopping*

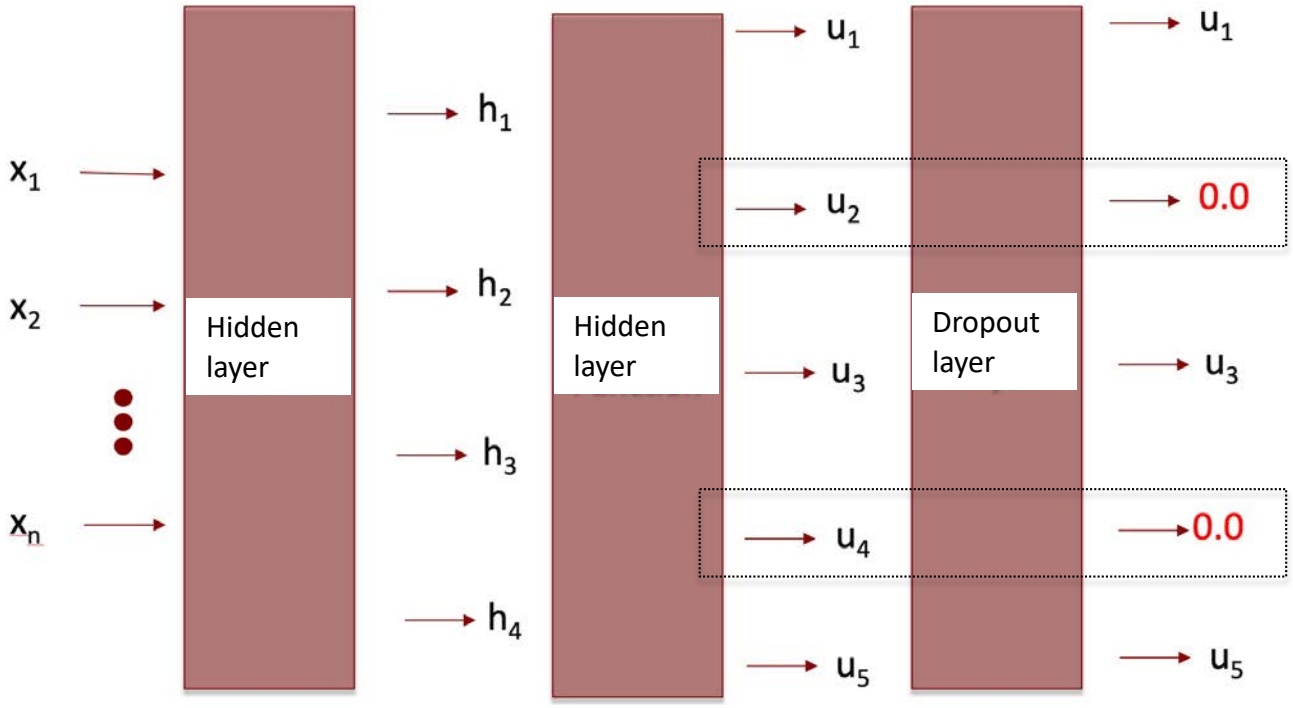
Stop the training early before the training loss is minimized by monitoring the loss on a validation dataset.



We will cover this in Lecture 4

Regularization strategy: *Dropout*

Randomly zero out the output from some of the nodes (typically 50% of the nodes) in a hidden layer (implemented as a "dropout layer" in Keras)



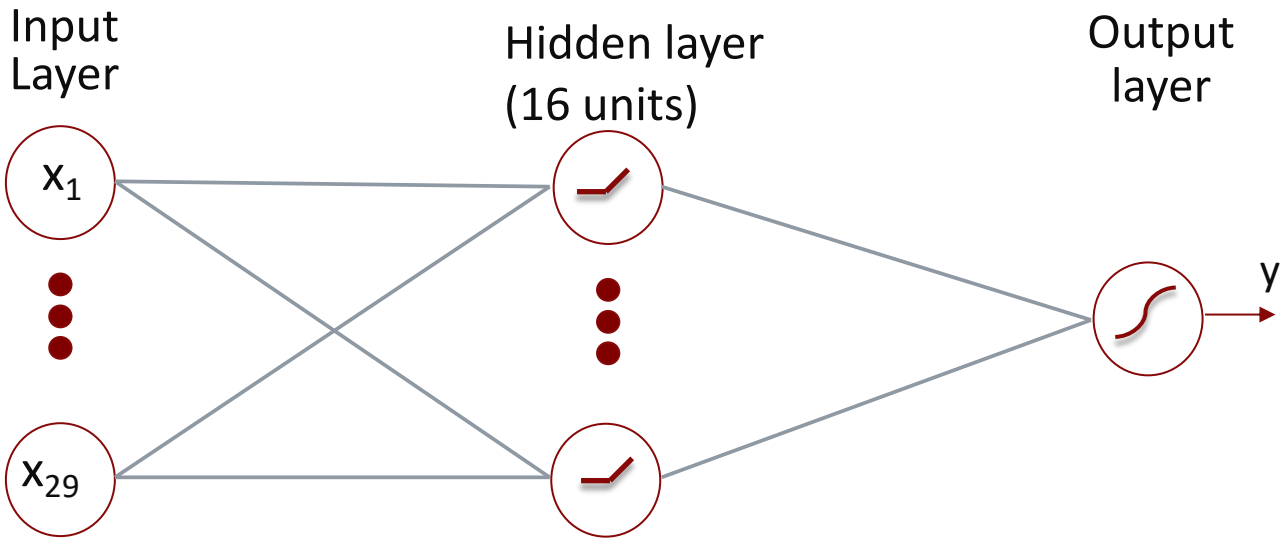
Summary: Creating and training a DNN from scratch

- We get the data ready
- We design i.e., “lay out” the network
  - Choose the number of hidden layers and the number of ‘neurons’ in each layer
  - Pick the right output layer based on the type of the output (more on this shortly)
- We pick
  - An appropriate loss function based on the type of the output (more on this shortly)
  - An optimizer from the many SGD flavors that are available and a “good” learning rate
- We decide on a regularization strategy
- We set things up in Keras/Tensorflow and start training!

Training the Model

ECE116 lecture section

Let's revisit the Neural Model for Heart Disease Prediction we designed previously




```
input = keras.Input(shape=29)
h = keras.layers.Dense(16, activation="relu")(input)
output = keras.layers.Dense(1, activation="sigmoid")(h)
model = keras.Model(input, output)
```

Let’s train this model!

Training Checklist

- We get the data ready (will cover in the colab)

- We design i.e., "lay out" the network 1 hidden layer with 16 ReLU neurons
  - Choose ***the number of hidden layers*** and ***the number of 'neurons' in each layer***
  - Pick the ***right output layer*** based on the type of the output Sigmoid

- We pick
  - An appropriate ***loss function*** based on the type of the output _____
  - An ***optimizer from the many SGD flavors*** that are available

- We decide on a ***regularization strategy***

- We set things up in Keras/Tensorflow and start training!

Training Checklist

- We get the data ready (will cover in the colab)

- We design i.e., "lay out" the network 1 hidden layer with 16 ReLU neurons
  - Choose ***the number of hidden layers*** and ***the number of 'neurons' in each layer***
  - Pick the ***right output layer*** based on the type of the output Sigmoid

- We pick
  - An appropriate ***loss function*** based on the type of the output binary crossentropy
  - An ***optimizer from the many SGD flavors*** that are available

- We decide on a ***regularization strategy***

- We set things up in Keras/Tensorflow and start training!

Training Checklist

- We get the data ready (will cover in the colab)

- We design i.e., “lay out” the network 1 hidden layer with 16 ReLU neurons
  - Choose ***the number of hidden layers*** and ***the number of ‘neurons’ in each layer***
  - Pick the ***right output layer*** based on the type of the output Sigmoid

- We pick
  - An appropriate ***loss function*** based on the type of the output binary crossentropy
  - An ***optimizer from the many SGD flavors*** that are available“adam"

- We decide on a ***regularization strategy*** Early stopping

- We set things up in Keras/Tensorflow and start training!

Colab

Predicting Heart Disease

Before we start coding …

- Don’t worry if you don’t understand every detail of what we will do in class.

- But go through the Colab notebooks carefully later, play around with the code and make sure you understand every line

Colab General Instructions

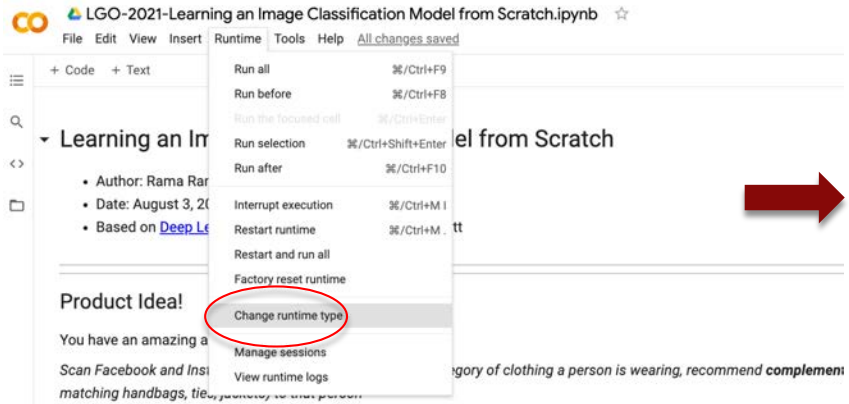
Step 1

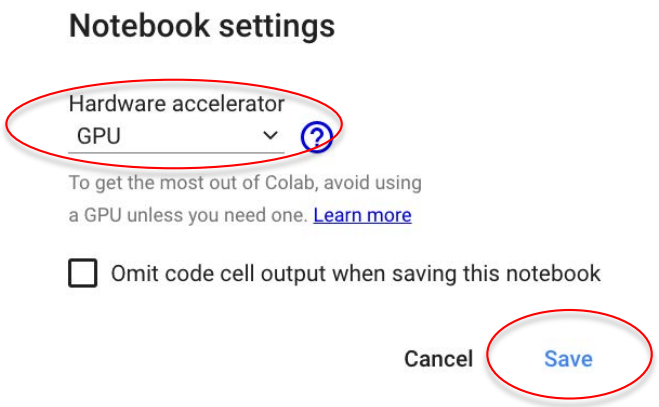
Make your own copy of the notebook



Step 2

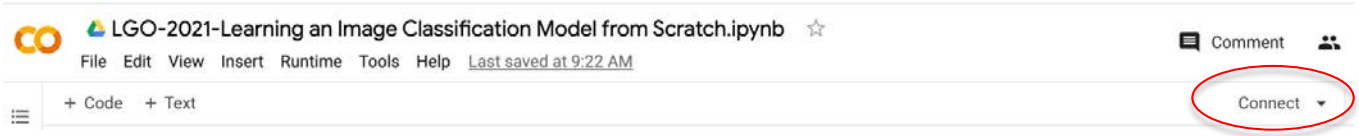
Request a GPU for your notebook*





Step 3

Start your notebook



You need to do steps 1 and 2 just the first time you use a notebook. From the second time onwards, jump to Step 3.

Source and license

This lecture adapts material from:

Vivek Farias and Rama Ramakrishnan, Hands-On Deep Learning,
MIT OpenCourseWare 15.773, Spring 2024.

Source:

ocw.mit.edu/courses/15-773-hands-on-deep-learning-spring-2024/

License: Creative Commons Attribution-NonCommercial-ShareAlike 4.0
creativecommons.org/licenses/by-nc-sa/4.0/

Adapted for ECE116 at UC San Diego, Fall 2026.
Changes include course framing, sequencing, omissions, and ECE116 branding.